Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Информатика

Лабораторная работа №4

Исследование протоколов, форматов обмена информацией и языков разметки документов

Вариант № 113

Выполнил:

Крахмальников Илья Артемович

Группа P3119

Проверил:

Осипов В. В.

Санкт-Петербург

2026

Содержание

Задание....3
Основные этапы вычисления....5
Составление исходного файла расписания....5
Задание 1....5
Деление текста....6
Парсинг в объект python....6
Парсинг объекта python в бинарный формат....6
Задание 2....7
Задание 3....8
Задание 4....9
Задание 5....10
Исходный код....11
Выводы по работе....12
Список литературы....13

Задание

1. Определить номер варианта как остаток деления на 132 своего идентификационного номера в ISU: например, 125598 / 132 = 16. В случае, если в обоих указанных днях недели нет занятий, то увеличить номер варианта на восемь. В случае, если занятий нет и в новом наборе дней, то продолжать увеличивать на восемь.
2. Изучить форму Бэкуса-Наура.
3. Изучить основные принципы организации формальных грамматик.
4. Изучить особенности языков разметки/форматов JSON, RON, HCL, YAML, TOML, INI, XML.
5. Понять устройство страницы с расписанием на примере расписания лектора: https://itmo.ru/ru/schedule/3/125598/raspisanie_zanyatiy.htm
6. Исходя из структуры расписания конкретного дня, сформировать файл с расписанием в формате, указанном в задании в качестве исходного. При этом необходимо, чтобы хотя бы в одной из выбранных дней было не менее двух занятий (можно использовать своё персональное). В случае, если в данный день недели нет таких занятий, то увеличить номер варианта ещё на восемь.
7. Обязательное задание (позволяет набрать до 50 процентов от максимального числа баллов БаРС за данную лабораторную). Написать программу на языке Python 3.x или любом другом, которая:
   - осуществляет парсинг и конвертацию исходного файла в бинарный объект (=десериализацию);
   - для решения задачи использует формальные грамматики; то есть ваш код должен уметь осуществлять парсинг и конвертацию любых данных, представленных в исходном формате, в данные, представленные в результирующем формате (как с готовыми библиотеками из дополнительного задания №2);
   - не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.
8. Дополнительное задание №1 (позволяет набрать +15 процентов от максимального числа баллов БаРС за данную лабораторную). Написать программу на языке Python 3.x или любом другом, которая:

- осуществляет парсинг и конвертацию бинароного объекта, полученного в обязательном задании, в новый формат (=сериализацию);
- для решения задачи использует формальные грамматики;
- не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.

9. Дополнительное задание №2 (позволяет набрать +10 процентов от максимального числа баллов БаРС за данную лабораторную).
    a) Найти готовые библиотеки, осуществляющие аналогичный парсинг и конвертацию файлов (десериализацию и сериализацию).
    b) Переписать исходный код и код из дополнительного задания №1, применив найденные библиотеки. Регулярные выражения также нельзя использовать.
    c) Сравнить полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.
10. Дополнительное задание №3 (позволяет набрать +20 процентов от максимального числа баллов БаРС за данную лабораторную). Переписать код из дополнительного задания №1, чтобы серализация происводилась в XML файл.
11. Дополнительное задание №4 (позволяет набрать +5 процентов от максимального числа баллов БаРС за данную лабораторную).
    a) Используя свою исходную программу из обязательного задания и программы из дополнительных заданий, сравнить стократное время выполнения парсинга + конвертации в цикле.
    b) Проанализировать полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.
12. Проверить, что все пункты задания выполнены и выполнены верно.
13. Написать отчёт о проделанной работе.
14. Подготовиться к устным вопросам на защите.

Основные этапы вычисления

Составление исходного файла расписания

На основе моего расписания на понедельник и субботу составляю файл schedule.ron

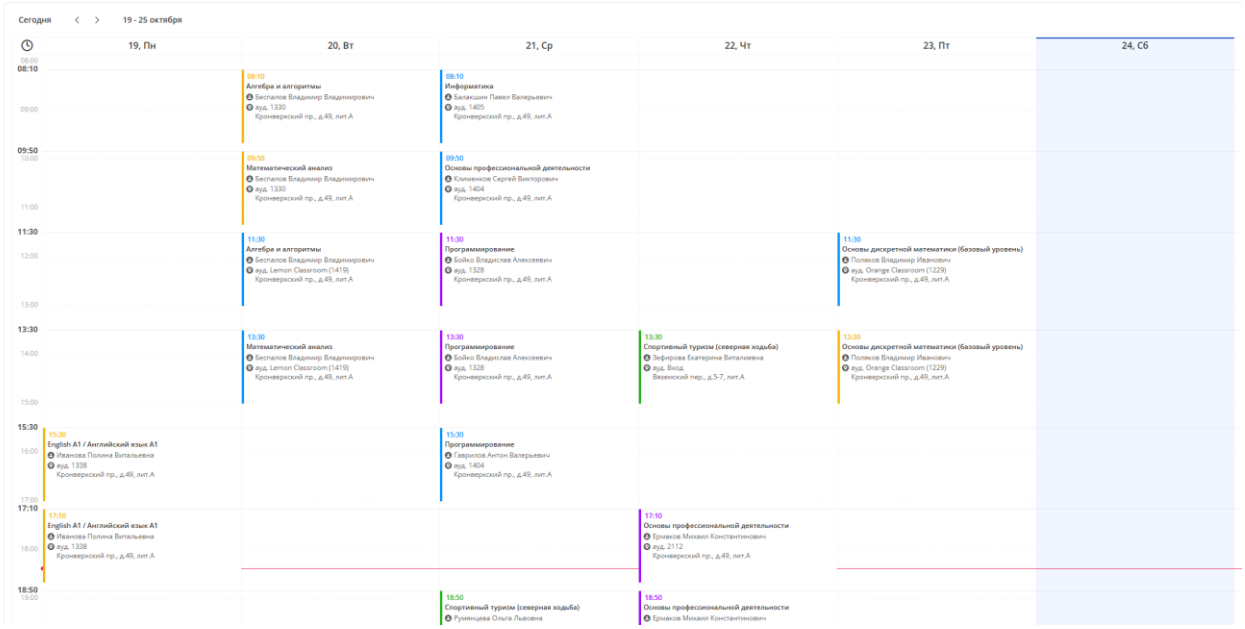


Рисунок 1.1 – Мое расписание

Исходный файл:

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule.ron

Задание 1

Нам необходимо перевести текст в бинарный формат, для этого мы разбиваем текст на части с помощью функции, а далее уже парсим в его части в объект python, состоящий из массивов и словарей. Затем преобразуем объект python в бинарный форма



Рисунок 1.2 – Логика преобразования из RON в бинарный формат

Деление текста

Особенностью языка разметки RON является:

- Пробельные символы не играют роли (кроме 1 символа для разделения между элементами)
- Комментарии могут быть только // или /* … */
- Строковые значения находятся в кавычках - “
- Объектами могут являться – массивы, словари, классы (в том числе анонимные – без названий)

Парсинг в объект python

После деления текста на части, можно рекурсивно собирать из них объект python, состоящий из массивов и словарей

```
<value> ::= <struct> | <dict> | <array> | <string> | <int> | <float> | <bool> | <ident>

<struct> ::= <ident> "(" <struct_fields> ")"
<struct_fields> ::= <empty> | <struct_pair> | <struct_pair> "," <struct_fields>
<struct_pair> ::= <ident> ":" <value>

<dict> ::= "{" <dict_fields> "}"
<dict_fields> ::= <empty> | <dict_pair> | <dict_pair> "," <dict_fields>
<dict_pair> ::= <string> ":" <value>

<array> ::= "[" <array_elements> "]"
<array_elements> ::= <empty> | <value> | <value> "," <array_elements>

<string> ::= "" <text> ""
<bool> ::= "true" | "false"

<ident> ::= <letter> <ident_tail>
<ident_tail> ::= <empty> | <letter> <ident_tail> | <digit> <ident_tail> | "_" <ident_tail>
```

Рисунок 1.3 - Нормальная форма Бэкуса-Наура

Парсинг объекта python в бинарный формат

\x00 – пустой

\x01 – тип булеан

\x02 – целое число

\x03 – дробное число

\x04 – строка

\x05 – массив

\x06 - словарь

Для преобразования целого числа используем 8 байт, при этом 1 для определения знака

to_bytes(length, byteorder, signed)

Для преобразования дробного числа или строки используем 8 байт для определения длины текста и остальное сразу в байтах

return b'\x04' + len(b).to_bytes(4, byteorder='big') + b

Массивы, словари рекурсивно преобразовываются, ссылаясь на ранее определенные форматы



Рисунок 1.4 – Пример преобразованного RON в бинарный формат

Исходный код:

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/ron.py

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/task1.py

Выходной файл:

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule.bin

Задание 2

Преобразуем сначала бинарный формат в объект python, затем уже в формат yaml

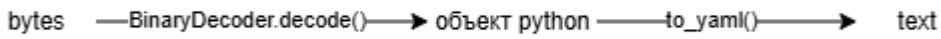


Рисунок 2.1 – Логика преобразования из бинарного формата в yaml

Исходный код:

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/yaml.py

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/task2.py

Выходной файл:

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule.bin

Задание 3

Для преобразования из RON в бинарный формат использовал библиотеку pyron

Поскольку библиотека не дает полного преобразования, а оставляет название объекта вне, то пришлось писать собственную функцию для приведения до объекта массивы-словари

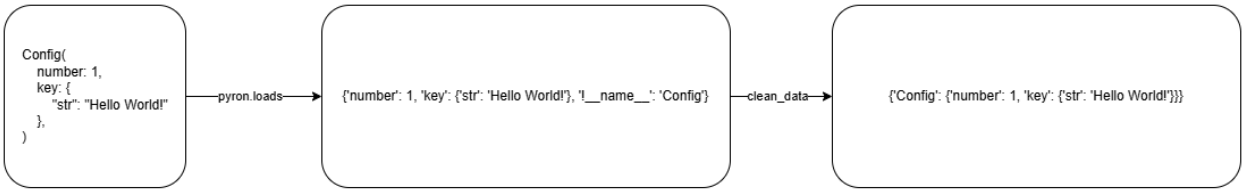


Рисунок 3.1 – Схема обработки RON -> объект python

Далее уже с помощью библиотеки pickle переводим в бинарное представление

Также с ее же помощью переводим из двоичного в объект python

Далее с помощью библиотеки yaml преобразовываем массивы-словари python в yaml формат

Сравнение полученных результатов

Сходства	Различия	
	Собственный код	Готовые библиотеки

Данные успешно распарсены	Чистые словари Python без лишних полей	pyron добавляет служебный ключ !__name__, требуется дополнительная очистка
Сохранение всей вложенности структуры файла	Для кодирования в бинарный формат мы сами контролируем, какой размер использовать (сейчас указано 8 байт на 1 число, из-за чего у нас выходит более большой файл .bin, чем у библиотеки)	Ограниченная возможность менять настройки кодирования, используются более сложные механизмы кодирования
	Только встроенные модули Python	Требуется установка сторонних библиотек - ron и yaml
	Чистый yaml	PyYAML по умолчанию генерирует нестандартные теги (!!python/tuple), требуется нормализация

Исходный код:

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/task3.py

Выходной файл:

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule_lib.bin

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule_lib.yaml

Задание 4

Парсинг данных в xml происходит аналогично yaml, используя рекурсивный спуск

Исходный код:

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/task4.py

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/code/xml.py

Выходной файл:

https://gitlab.se.ifmo.ru/124476/informatics/-/blob/master/lab4/data/schedule/schedule.xml

Задание 5

Переделал немного реализацию 1-4 задания, чтобы можно было запускать с помощью комнады python task[].py <название файла>

Далее импортировал задания 2 и 3 и запустил на итерациях 100, 1000, 5000, 10000

```
Разница:                0.45x
(.venv) PS C:\Users\User\IdeaProjects\itmo\informatic\labs\lab4\code> python task5.py schedule 100
Собственный код:        0.1321 сек.
Готовые библиотеки:     0.2403 сек.
Разница:                0.55x
(.venv) PS C:\Users\User\IdeaProjects\itmo\informatic\labs\lab4\code> python task5.py schedule 1000
Собственный код:        1.4977 сек.
Готовые библиотеки:     5.1218 сек.
Разница:                0.29x
(.venv) PS C:\Users\User\IdeaProjects\itmo\informatic\labs\lab4\code> python task5.py schedule 5000
Собственный код:        7.4758 сек.
Готовые библиотеки:     13.2400 сек.
Разница:                0.56x
(.venv) PS C:\Users\User\IdeaProjects\itmo\informatic\labs\lab4\code> python task5.py schedule 10000
Собственный код:        14.9374 сек.
Готовые библиотеки:     28.1257 сек.
Разница:                0.53x
```

Рисунок 5.1 – Результат работы

В среднем собственный код работает в 2 раза быстрее готовых библиотек

Работа библиотек скорее всего сильно замедляются из-за собственной функции clean_data, которую я использую для достроения объекта python после использования библиотеки ron.

Из-за этого происходит неоднократный обход всего объекта в отличие от моего кода.

Исходный код

https://gitlab.se.ifmo.ru/124476/informatics/-/tree/master/lab4/

Выводы по работе

В результате работы я изучил языки разметки ron, yaml, xml. Также научился производить сериализацию и десериализацию, перевод из одних форматов файлов в другие.

Список литературы

1. python-ron 0.2.7. Python bindings for Rusty Object Notation. – URL: https://pypi.org/project/python-ron/ (дата обращения: 25.10.2026)

2. Формальная грамматика // Википедия. - URL: https://ru.wikipedia.org/wiki/Формальная_грамматика (дата обращения: 25.10.2026)

3. Форма Бэкуса — Наура // Википедия. - URL: https://ru.wikipedia.org/wiki/Форма_Бэкуса_—_Наура (дата обращения: 25.10.2026)